

Linked Lists

Learning Outcomes

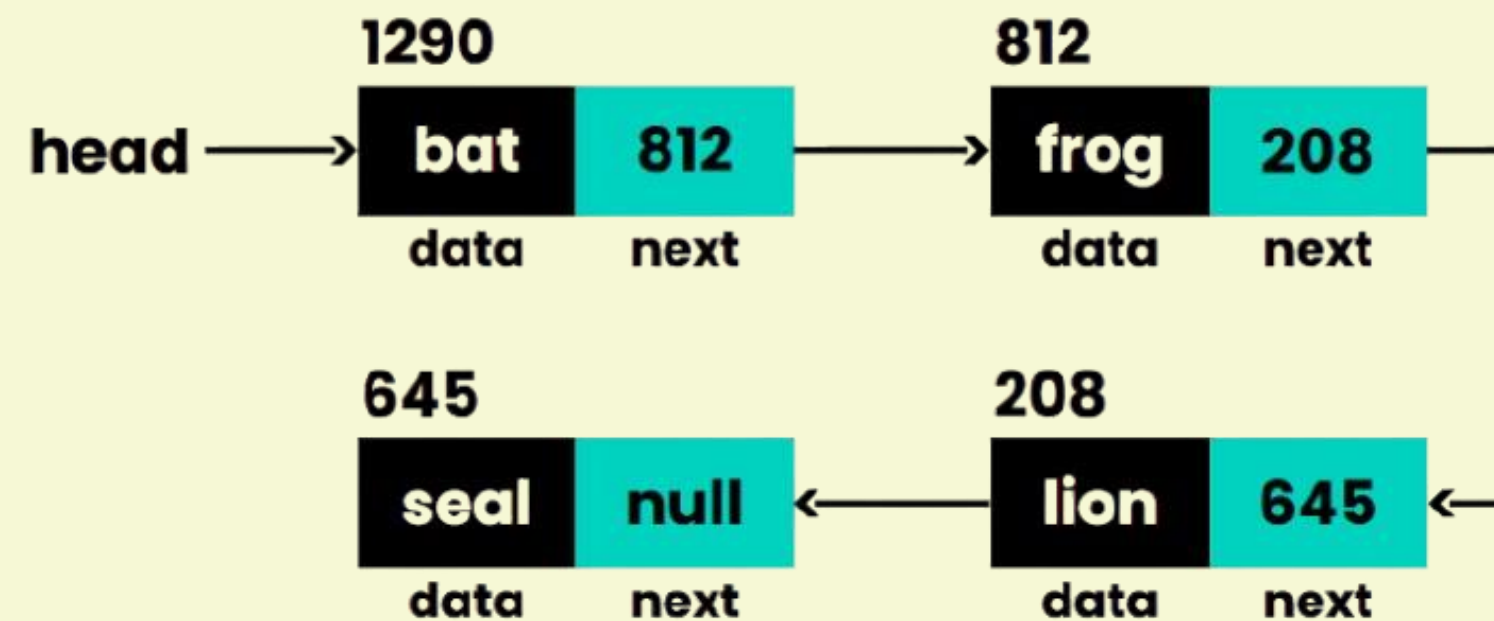
- LO 2.1 Explain memory representation of linked list*
- LO 2.2 Implement insertion, deletion, and search operations in linked list*
- LO 2.3 Analyze time complexity in linked list operations*
- LO 2.4 Diagnose linked list-related implementation errors*

Setting the Expectations

PREREQUISITES

- Understood the **intermediate programming concepts**—pointers, pointer arithmetic, dereferencing, structures and classes, and memory allocation.
- Utilized **pointers** in **C/C++**.

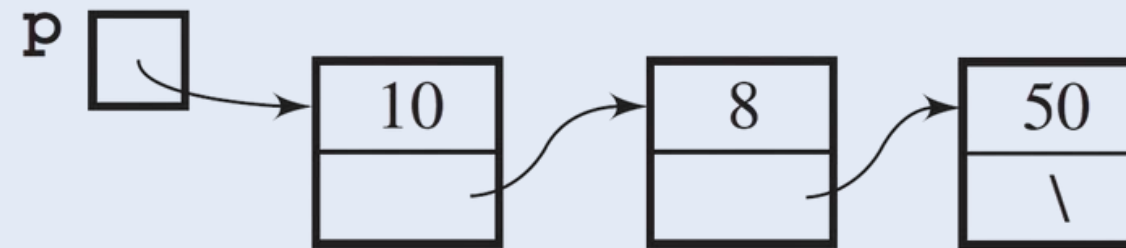
What's a Linked List?



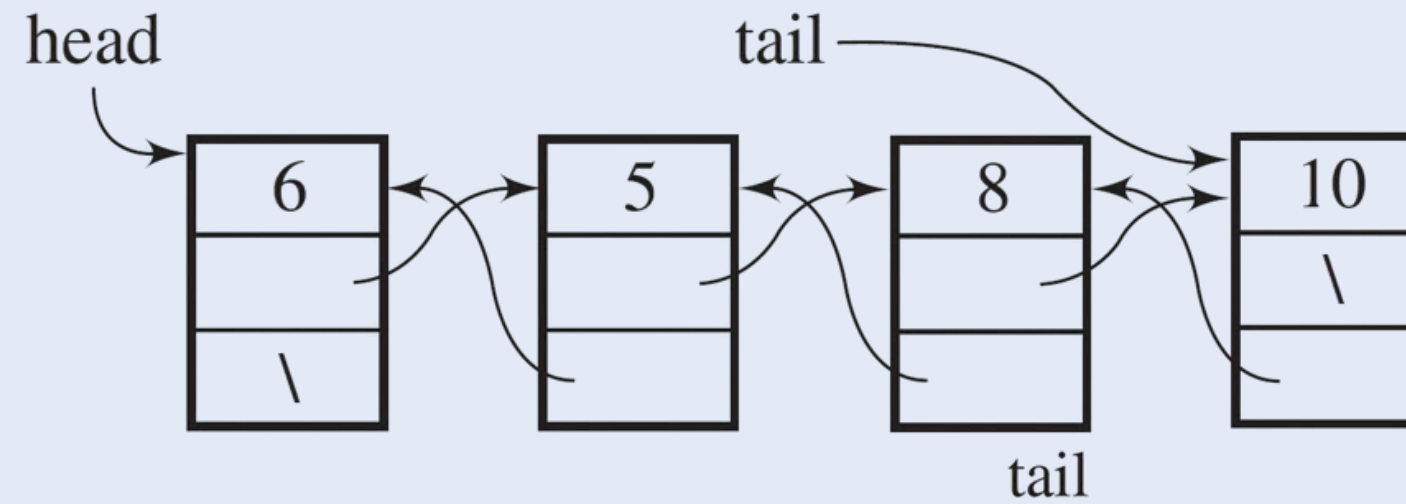
Nodes scattered in memory chained together

Types of Linked List

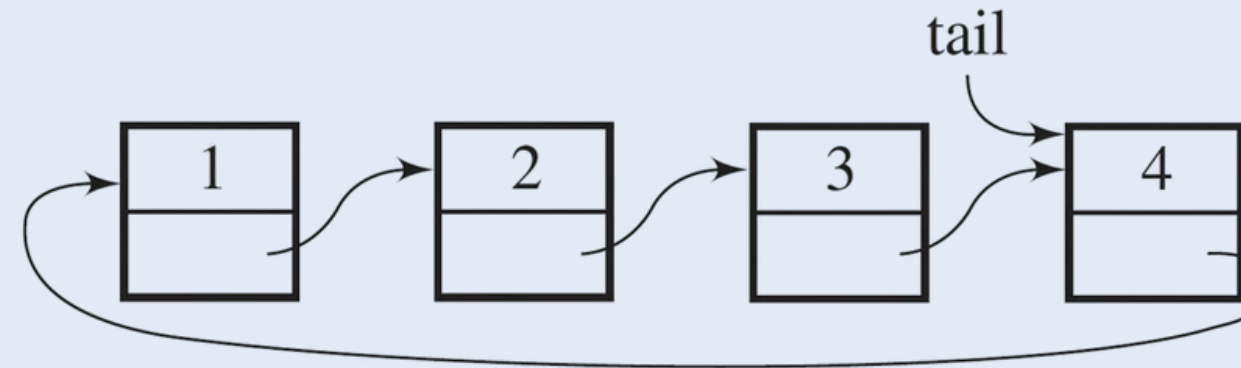
Singly Linked List



Doubly Linked List



Circular Linked List



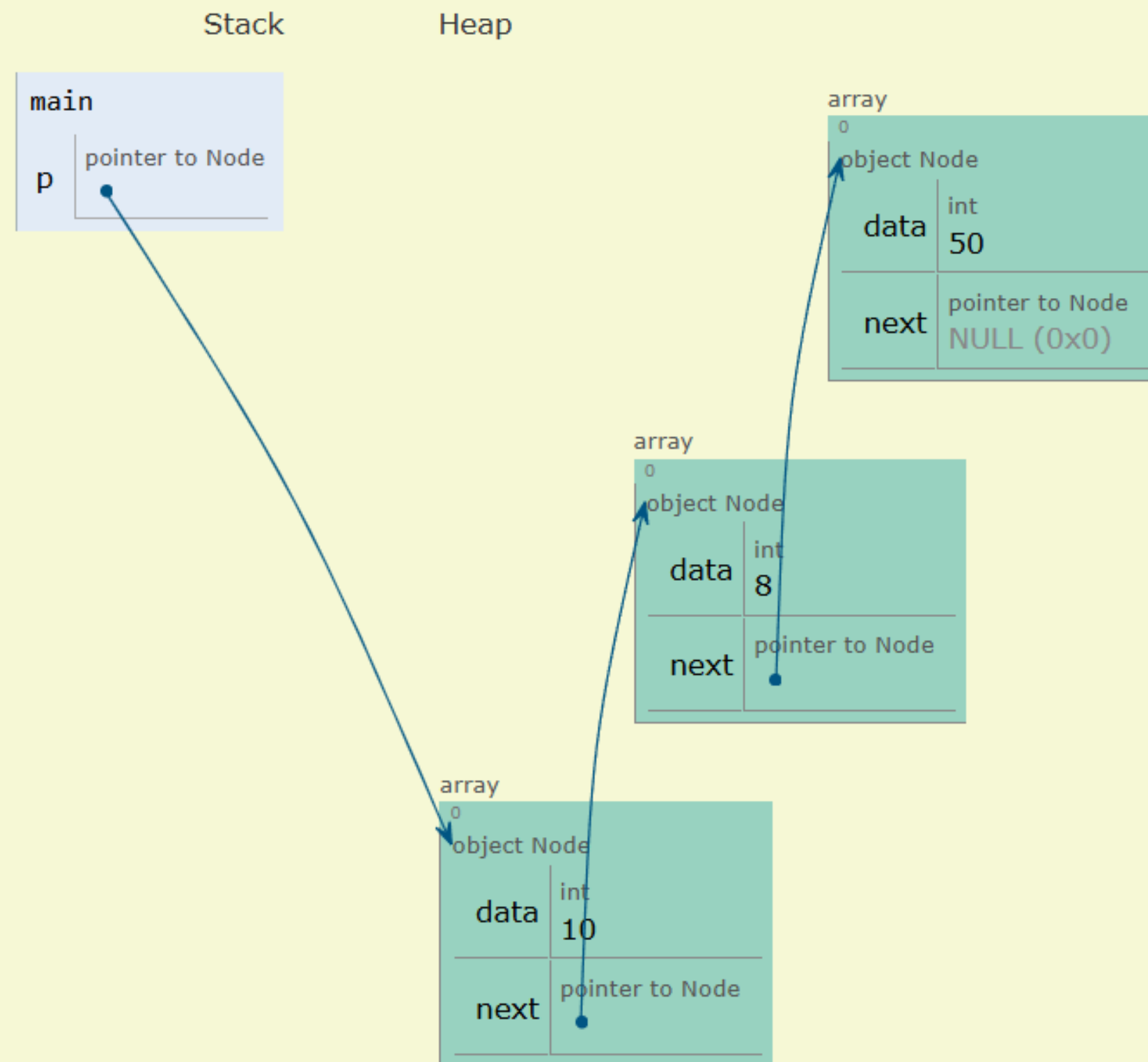
Basic Unit: Nodes

```
1 class Node {  
2     public:  
3         int data;  
4         Node* next;  
5 };
```

```
1 class DNode {  
2     public:  
3         int data;  
4         DNode* next;  
5         DNode* prev;  
6 };
```

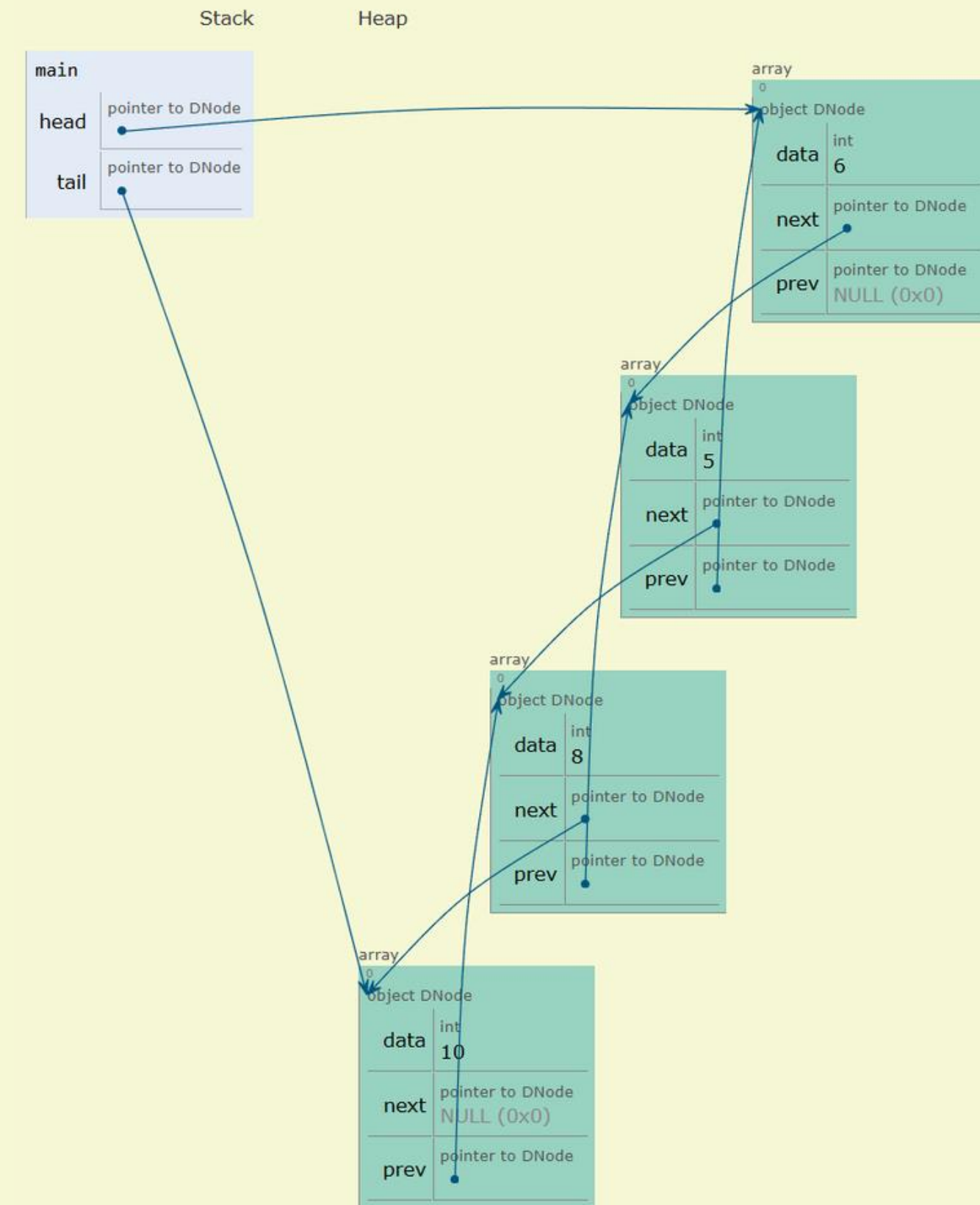
Singly Linked List

```
1 class Node {  
2     public:  
3         int data;  
4         Node* next;  
5 };  
6  
7 int main() {  
8     Node* p = new Node{10,  
9                     new Node{8,  
10                    new Node{50, nullptr}  
11                }  
12            };  
13     return 0;  
14 }
```



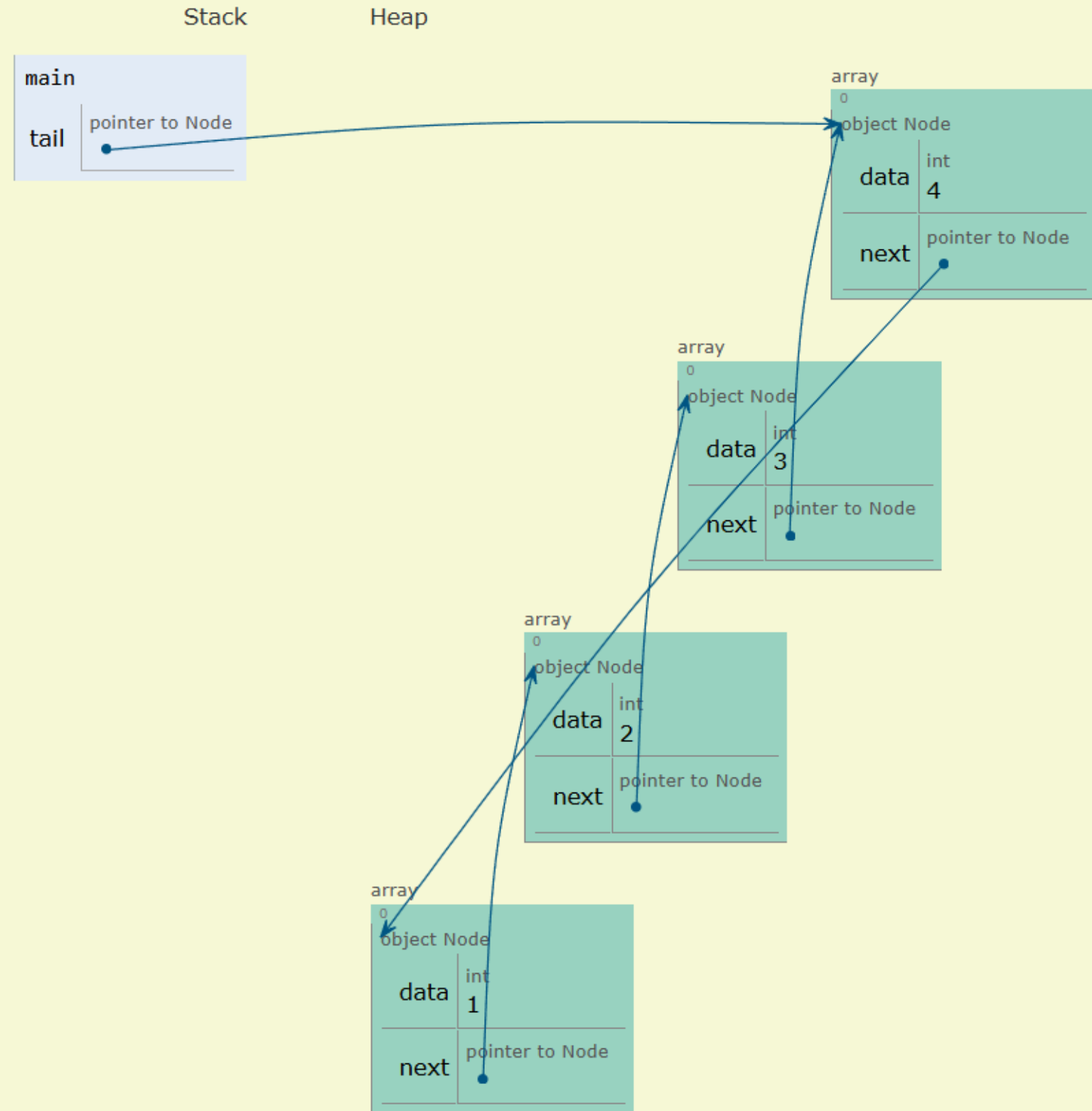
Doubly Linked List

```
1 class DNode {
2     public:
3         int data;
4         DNode* next;
5         DNode* prev;
6 };
7
8 int main() {
9     DNode* head = new DNode {6, nullptr, nullptr};
10    head->next = new DNode {5, nullptr, head};
11    head->next->next = new DNode {8, nullptr, head->next};
12    DNode* tail =
13        head->next->next->next =
14        new DNode {10, nullptr, head->next->next};
15    head->next->prev = head;
16    head->next->next->prev = head->next;
17    tail->prev = head->next->next;
18    return 0;
19 }
```

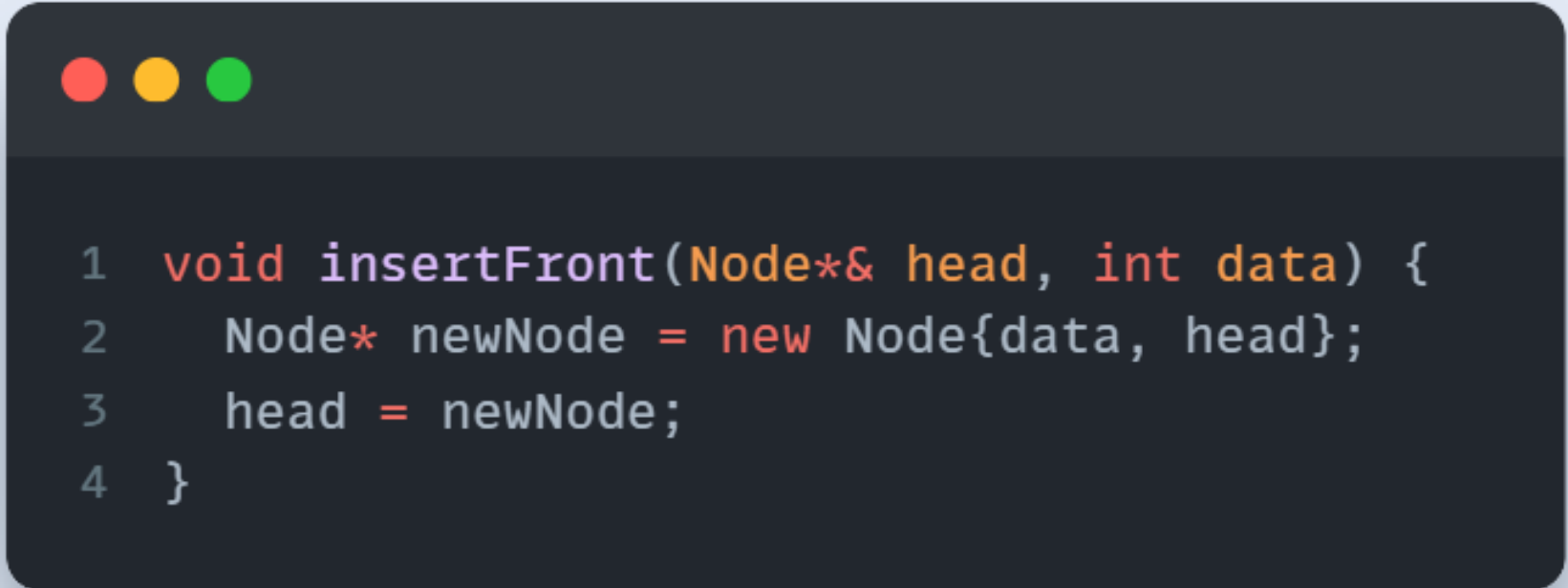


Circular Linked List

```
1 class Node {
2     public:
3         int data;
4         Node* next;
5 };
6
7 int main() {
8     Node* tail = new Node{4, nullptr};
9     tail->next = new Node{1,
10                        new Node{2,
11                        new Node{3, tail}}
12                };
13 };
14 return 0;
15 }
```



Singly Linked List: Insert



```
1 void insertFront(Node*& head, int data) {  
2     Node* newNode = new Node{data, head};  
3     head = newNode;  
4 }
```

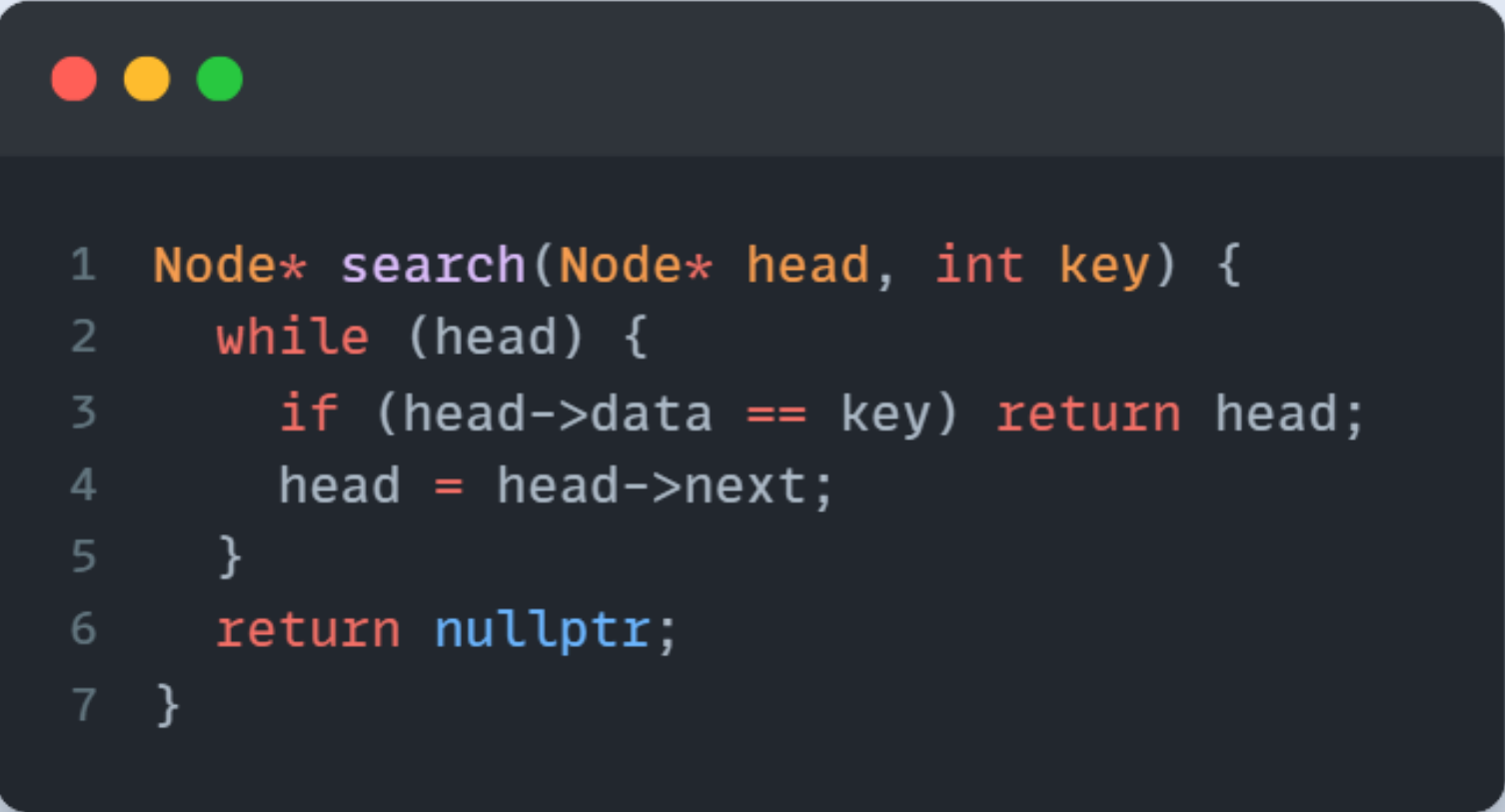
Singly Linked List: Insert

```
1 void insertRear(Node*& head, int data) {
2     Node* newNode = new Node{data, nullptr};
3     if(head == nullptr)
4         head = newNode;
5     else
6     {
7         Node* p = head;
8         while(p->next)
9             p = p->next;
10        p->next = newNode;
11    }
12 }
```

Singly Linked List: Delete

```
1 void deleteNode(Node*& head, int key) {  
2     Node* temp = head, *prev = nullptr;  
3     while (temp && temp->data != key) {  
4         prev = temp;  
5         temp = temp->next;  
6     }  
7     if (!temp) return;  
8     if (prev) prev->next = temp->next;  
9     else head = temp->next;  
10    delete temp;  
11 }
```

Singly Linked List: Search



```
1 Node* search(Node* head, int key) {  
2     while (head) {  
3         if (head->data == key) return head;  
4         head = head->next;  
5     }  
6     return nullptr;  
7 }
```

Doubly Linked List: Insert

```
1 void insertTail(DNode*& head, DNode*& tail, int data) {  
2     DNode* newNode = new DNode{data, nullptr, tail};  
3     if (!head) head = newNode;  
4     else {  
5         tail->next = newNode;  
6         newNode->prev = tail;  
7     }  
8     tail = newNode;  
9 }
```

Doubly Linked List: Insert

```
1 void insertBeforeX(DNode*& head, DNode*& tail, DNode*& x, int data) {
2     // If x is nullptr, no insertion
3     if (x == nullptr) return;
4
5     // Create the new node
6     DNode* newNode = new DNode{data, nullptr, nullptr};
7
8     // Insert before the head node
9     if (x == head) {
10         newNode->next = head;
11         head->prev = newNode;
12         head = newNode;
13     }
14     // Insert before a non-head node
15     else {
16         newNode->prev = x->prev;
17         newNode->next = x;
18         x->prev->next = newNode;
19         x->prev = newNode;
20     }
21 }
```

Doubly Linked List: Delete

```
1 void deleteX(DNode*& head, DNode*& tail, DNode* x) {
2     if (head == nullptr || x == nullptr) return;
3
4     if (x == head)
5     {
6         head = x->next;
7         if (head != nullptr) head->prev = nullptr;
8         else tail = nullptr;
9     }
10    else if (x == tail)
11    {
12        tail = x->prev;
13        tail->next = nullptr;
14    }
15    else
16    {
17        x->prev->next = x->next;
18        x->next->prev = x->prev;
19    }
20
21    delete x;
22 }
```

Doubly Linked List: Search

```
1 DNode* search(DNode*& head, DNode*& tail, int key) {
2     DNode* p = head;
3     while (p != nullptr) {
4         if (p->data == key) return p;
5         p = p->next;
6     }
7     return nullptr;
8 }
```

Circular Linked List: Insert

```
1 void insertFront(Node*& tail, int data)
2 {
3     if(tail) tail->next = new Node{data, tail->next};
4     else {
5         tail = new Node{data, nullptr};
6         tail->next = tail;
7     }
8 }
```

Circular Linked List: Insert

```
1 void insertRear(Node*& tail, int data)
2 {
3     if(tail){
4         tail->next = new Node{data, tail->next};
5         tail = tail->next;
6     }
7     else {
8         tail = new Node{data, nullptr};
9         tail->next = tail;
10    }
11 }
```

Circular Linked List: Delete

```
1 void deleteFront(Node*& tail) {
2     if(!tail) return;
3     if(tail->next == tail)
4     {
5         delete tail;
6         tail = nullptr;
7         return;
8     }
9     Node* temp = tail->next;
10    tail->next = temp->next;
11    delete temp;
12 }
```

Circular Linked List: Search

```
1 Node* search(Node* tail, int key) {
2     if(tail)
3     {
4         Node* p = tail;
5         do{
6             if (p->data == key) return p;
7             p = p->next;
8         } while(p != tail);
9     }
10    return nullptr;
11 }
```

Complexity of Linked List Operations

OPERATION	TIME COMPLEXITY	WHY?
Singly Linked List		
Insert in front	$O(1)$	Just update head pointer. No traversal needed.
Insert in rear	$O(n)$	Must traverse entire list to find the tail.
Delete	$O(n)$	Worst-case: Search entire list for the node.
Search	$O(n)$	Linear scan; no random access.

Complexity of Linked List Operations

OPERATION	TIME COMPLEXITY	WHY?
Doubly Linked List		
Insert at tail	$O(1)$	Tail pointer allows direct access.
Insert before X	$O(1)$	Given X, update X->prev and neighbors.
Delete X	$O(1)$	Given X, update X->prev and X->next.
Search	$O(n)$	Still requires linear traversal.

Complexity of Linked List Operations

OPERATION	TIME COMPLEXITY	WHY?
Circular Linked List		
Insert at front	$O(1)$	Just update tail's next pointer.
Insert at rear	$O(1)$	Tail pointer allows instant access.
Delete front	$O(1)$	Just update tail's next pointer.
Search	$O(n)$	Linear scan; loop until node after tail.

Debugging Linked List Errors

```
1 void deleteNode(Node*& head, int key) {  
2     Node* temp = head;  
3     Node* prev = nullptr;  
4  
5     while (temp && temp->data != key) {  
6         prev = temp;  
7         temp = temp->next;  
8     }  
9  
10    if (!temp) return;  
11  
12    if (prev)  
13        prev->next = temp->next;  
14    else  
15        head = temp->next;  
16 }
```

- ✗ Memory leak
- ✗ Forgetting to deallocate the node
- ✓ Add `delete temp;` before the function ends

Debugging Linked List Errors

```
1 void deleteLast(Node* head) {  
2     if (!head || !head->next) return;  
3  
4     Node* curr = head;  
5     while (curr->next->next) curr = curr->next;  
6  
7     delete curr->next;  
8 }
```

- ✗ Dangling pointer
- ✗ Cannot terminate during traversal
- ✓ Set pointer to null after deletion

Debugging Linked List Errors

```
1 void printCircularLL(Node* head) {  
2     Node* current = head;  
3  
4     while (current != nullptr) {  
5         cout << current->data << " ";  
6         current = current->next;  
7     }  
8 }
```

- ✗ Infinite loop
- ✗ There should be no null pointer in a circular linked list if it is not empty
- ✓ Use do while and halting condition should be if it returns back to the first node printed

Applications

Data structures - Unbounded stack and queues,
Chained hash tables, Graph adjacency lists, etc.

Serves as containers for anything that increases and
decreases arbitrarily at runtime (e.g. *music playlists*,
process threads, browser tabs)

Limitations

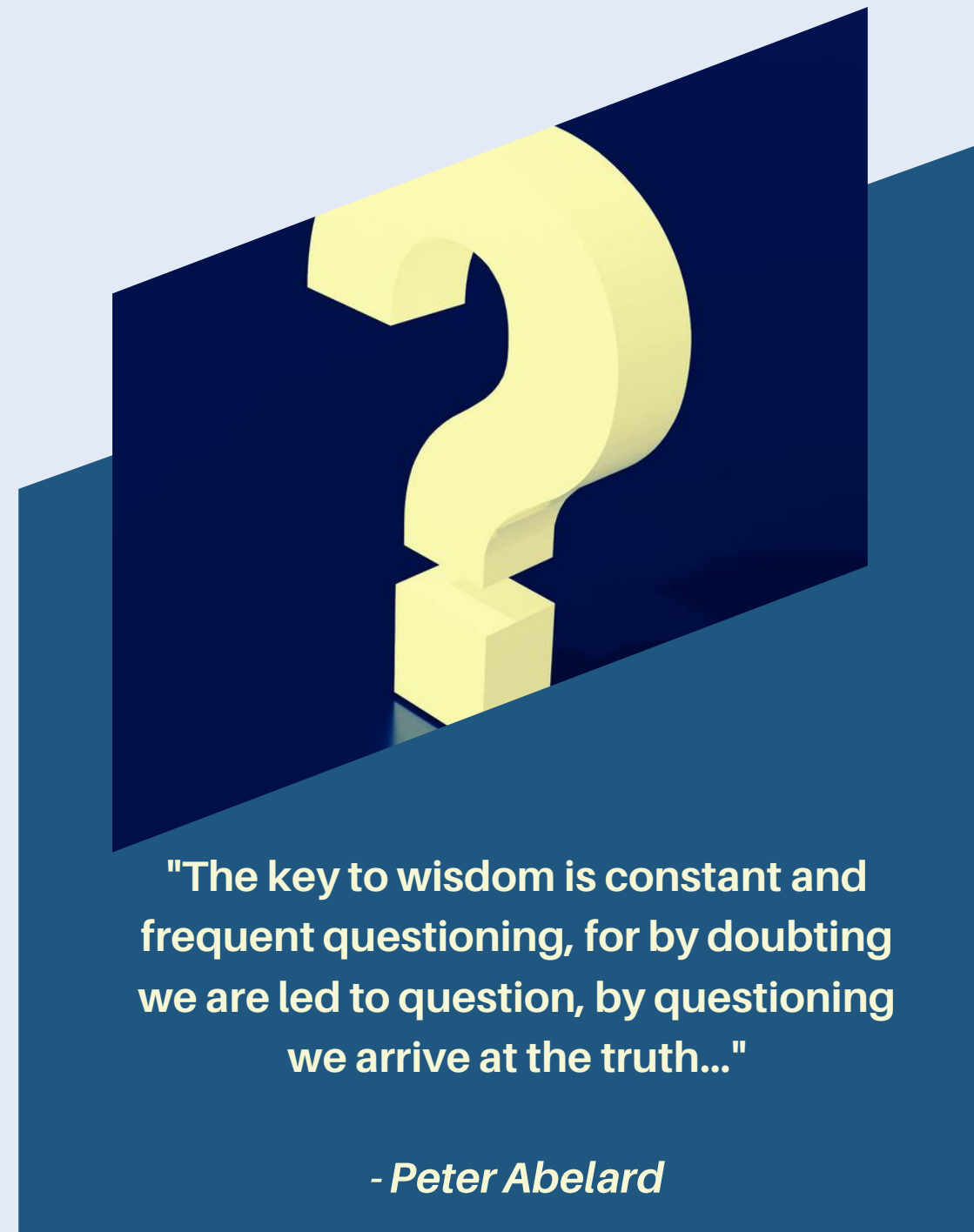
Memory overhead

No random access → Traversal needed

Nodes scattered randomly in memory → High cache misses

Golden Rule

*“Insertion and deletion = fast.
Space and access = costly.
Choose your priorities.”*



OPEN FORUM Q/A

for Module 2